FACTORYMANAGER AI

The AI Plant Manager for MSME Manufacturing

Product & Build Specification — Development Team Edition

How the product works, layer by layer, from data entry to AI decisions

Version 5 | Confidential | For the development team

How to Read This Document

This document explains WHAT the product does and WHY, not HOW to build it technically. There are no software-architecture decisions, no database designs, and no technology choices here — those belong to the technical lead and the build team.

It is written so that anyone — a plant owner, a supervisor, or a new engineer joining the build team — can read it from start to finish and fully understand the product. This edition is written specifically for the development team: every screen, formula, data item and behaviour needed to build the launch product is defined, and the final section lists exactly what the team needs on hand to start.

Hardware is part of the product — the build starts software-only

Hardware (IoT sensors that count cycles and detect downtime automatically) is part of the product from the beginning. The data layer is designed from day one so a sensor feed plugs into the exact same slot a supervisor's thumb occupies today — nothing above the data layer ever needs to change when hardware arrives.

> **What the build team focuses on now: the software-only version.** Which IoT device is used for which class of machine (CT clamps, controller taps, pulse counters, vision counters, etc.) is a decision made later, plant by plant, once the software has proven value. The team builds the software so that automatic data can be ingested through the same data contract when that decision is made — but no firmware, calibration or device work happens in this phase.

Everything in the launch version runs on data a supervisor can enter on a phone or tablet in under 30 seconds per event. If a launch feature would require a sensor to work, it is not in this version — it waits for the hardware phase.

Document Map

- What the product is, in plain language (Section 1)
- Why the build is software-first, with hardware designed in from the start (Section 2)
- The six layers of the product, each explained with examples — including the Job Order spine and traceability (Section 3)
- Exactly what data the system needs — upload-first onboarding, the three costing modes, and daily data (Section 4)
- The Actions & PDCA engine — how the morning meeting routes issues to PDCA/RCA by severity, and closes them into Factory Memory (Section 5)
- A full day in the life of a plant using the product (Section 6)
- The screens — role-based, money-first for the owner (Section 7)
- KPIs and the complete formula reference for the build team (Section 8)
- Build order — what to build first, second, third, and what NOT to build (Section 9)
- Implementation model — the transformation framing, and the onboarding consultant / support roles (Section 10)
- Why owners buy, why they say no, and the product moat (Section 11)
- Risk register — what could go wrong and what we design against it (Section 12)
- The hardware upgrade path and the long-term expansion roadmap (Section 13)

- Operational design — where it physically starts, tablet placement, stage monitoring, mid-job part changes, cost collection at scale, and AI Coach inputs (Section 14)
- The onboarding template pack — the exact seven templates, column by column (Section 15)
- What the development team needs to start — data, formulas, templates and open decisions (Section 16)

Section 1 — What Is FactoryManager AI?

A note on the name

This product was previously called Lean AI DMS. The name changed because 'DMS' describes only one module — production entry, downtime, OEE — and invites comparison with a crowded category of digital DMS and MES tools. What the customer actually buys is an AI plant manager: something that watches the whole plant, converts everything into money, decides what to fix first, and drives the fix to closure. The Daily Management System remains the name of the core data module inside the product; FactoryManager AI is the product.

The problem, in one story

Imagine a typical 20-machine auto-component plant in Hosur. Yesterday, Machine CNC-3 stood idle for 47 minutes because a tool broke. Line 2 waited 31 minutes for material that Stores didn't stage in time. The night-shift supervisor wrote '95% target achieved' in the production register — a number he padded slightly, as he does every day, because nobody can check.

This morning, the owner walks into the morning meeting. Nobody mentions the tool breakage. Nobody knows it cost the plant ₹3,290. The material wait is blamed on 'the supplier' with no follow-up. The meeting ends. The same problems will happen again tomorrow. Money is leaving the plant silently, every shift, and nobody is counting it.

FactoryManager AI is a system that makes every loss visible, in rupees, the moment it happens — and then tells the plant exactly what to fix first to get that money back.

The one-sentence pitch to an owner: "We find ₹25–45 lakh of hidden, recoverable profit inside a typical 20-machine plant every year — and show you, every morning, the one action that recovers the most of it." This figure comes from real shopfloor observations across Hosur, Ambattur and Coimbatore plants — never inflate it. The demo flow is always: Machine → Money → Action.

What it does, in five sentences

- Every shift, the supervisor and quality inspector enter what actually happened — production counts, downtime events with reasons, and rejections — through simple screens that take seconds to fill.
- The system instantly converts every minute of downtime and every rejected part into a rupee figure, using cost rates entered once during setup.
- At shift close, the data locks permanently — nobody can edit it afterwards — and the owner receives a report on their phone: how much money was lost, on which machine, for which reason.
- Every morning before the shift starts, the system's AI reads the plant's history and sends the supervisor three specific instructions: which machine to check, what to do, who should do it, and how much money it will save.
- Over time the system remembers everything — every breakdown, every fix, every defect — and anyone in the plant can ask it questions over WhatsApp and get an answer grounded in their own plant's data.

Who uses it

User	What they do in the system	How often
Plant Owner / MD	Receives daily loss report on phone. Sees monthly trends. Approves overrides.	Reads daily, 2 min

User	What they do in the system	How often
Shift Supervisor	Starts the shift, enters hourly production counts, logs downtime with reasons, closes the shift.	Throughout each shift
Quality Inspector	Logs good parts and rejections with defect codes at batch completion.	2–4 times per shift
Maintenance In-charge	Receives escalation alerts. Logs what was done to fix breakdowns.	When events occur
Stores In-charge	Receives material-shortage alerts. Confirms material staging.	When events occur
Onboarding consultant	Sets up cost parameters, machine list, reason codes; trains the team in week 1.	Once per plant + reviews

Section 2 — Why Software-First, With Hardware Designed In

The long-term product includes sensors: automatic cycle counting and downtime detection per machine. Automatic data is tamper-proof data, and it is the destination. The build simply starts software-only, for three reasons:

- **Time and cost:** hardware means firmware, calibration per machine type, Wi-Fi problems in Tamil Nadu industrial estates, voltage-spike filtering, and roughly ₹1.4 lakh of hardware per 20-machine plant before a single rupee of value is proven.
- **Risk stacking:** launching on hardware tests two unproven things at once — whether plants will pay for this, AND whether our hardware works in their conditions. If a pilot fails, we won't know which one failed.
- **It is not needed to prove value:** the value of this product is the cost intelligence, the morning discipline, and the AI coaching. All of that runs on data — it does not care whether the data came from a sensor or a supervisor's thumb.

> **Design consequence for the build team:** the data layer must treat "who produced this data point" as interchangeable. A production count or a downtime start/stop must enter through one internal contract, whether typed by a supervisor today or emitted by a sensor later. Get this right now and the hardware phase is a data-source swap, not a rebuild.

How the software-only version replaces the sensors

Every automatic function from the hardware plan has a manual equivalent that works today:

Hardware version (later)	Software-only version (launch)	Trade-off
CT clamp / counter counts every cycle automatically	Supervisor enters production count at fixed intervals (hourly or per batch) on a tablet/phone	Counts are approximate to the hour, not the second — good enough for cost analysis
Downtime auto-detected when no cycle for 1.5× cycle time	Supervisor taps 'Machine Stopped', picks a reason from a dropdown, taps 'Running' when it restarts	Relies on discipline — solved by the Shift Closure Lock and visible cost ticker (Section 3)
Sensor timestamp is tamper-proof	Data locks permanently at shift close; edits before close are logged; owner sees an audit trail	A supervisor could under-report during the shift — mitigated by quality cross-checks and owner spot audits
IoT gateway buffers data offline	App works offline on the tablet and syncs when internet returns	Same capability, no extra hardware

The honest weakness, and how we handle it

Manual data can be manipulated — that is the exact disease this product exists to cure. Three design choices keep manual data honest:

- **The Shift Closure Lock.** A supervisor cannot close their shift while any downtime event has no reason, any production entry is missing, or any escalation is unresolved. Closing the shift is how they go home — so the data gets filled.

- **Cross-verification.** The quality inspector's good-parts count is entered independently of the supervisor's production count. If the two don't reconcile, the system flags it to the owner automatically. Two people would have to collude to fake the numbers.
- **Cost visibility changes behaviour.** When the whole shift can see 'Line 3: ₹8,400 lost today' on a screen, hiding downtime stops being possible and starts being pointless — the missing production shows up in the count anyway.

Positioning for the owner: "Start with your people entering the data — you'll recover losses within the first month. When you're ready, add sensors and the same system becomes fully automatic." Hardware becomes an upsell to a happy customer, not a barrier to the first sale.

Section 3 — The Six Layers of the Product

The product is built like a building: each layer stands on the one below it. We build from the bottom up. The bottom two layers are the launch product. The plugin layer builds in parallel (Layer 5). Benchmark waits for scale.

Layer	Name	One-line description	When we build it
1	Factory Core	Captures what happened: production, downtime, quality, shift closure	Launch (MVP)
2	Factory Brain	Converts events into rupees and ranks what to fix first	Launch (MVP)
3	Factory Copilot	Lets anyone ask questions over WhatsApp and get answers from their own data	Day 1 (factual); pattern answers unlock as data grows
4	Factory Memory	Remembers every problem and fix forever; answers 'have we seen this before?'	Recording from Day 1; retrieval value compounds
5	Factory Plugins	Compliance/Audit AI, Kaizen AI, Skill Matrix AI — built in parallel on the same Core data	In parallel from the start (see 3.5)
6	Industry Benchmark	Compares a plant against similar plants (anonymised)	At 40–50 live plants

Layer 1 — Factory Core (the data-capture layer)

What it is

Factory Core is the foundation: a set of simple screens through which the plant records everything that happens during a shift. Nothing intelligent happens here — this layer's only job is to make recording reality so easy and so enforced that the data is complete and trustworthy.

What gets captured, and by whom

Event	Who enters it	When	Time to enter
Shift start — select job order, part number, machine, operator	Supervisor	Start of shift	30 seconds
Production count	Supervisor (or operator)	Every hour, or at batch completion	15 seconds
Downtime start — machine + reason from dropdown	Supervisor	When a machine stops	20 seconds
Downtime end	Supervisor	When the machine restarts	5 seconds (one tap)
Quality — good parts + rejections with defect codes	Quality Inspector	At batch completion	60 seconds
Shift closure — confirm all data, resolve open items	Supervisor	End of shift	2–3 minutes

Example scenario

10:14 AM. CNC-3's tool snaps. The operator calls the supervisor. The supervisor pulls out the shared tablet, taps CNC-3 on the machine grid, taps 'Machine Stopped', selects 'Tool Breakage' from the reason list, and puts the tablet down. Total time: 20 seconds. From that moment, a red timer runs on CNC-3's tile — and next to it, a rupee

counter is climbing. At 11:01 AM the new tool is fitted; the supervisor taps ‘Running’. The event is recorded: CNC-3, Tool Breakage, 47 minutes, 10:14–11:01.

The reason-code list — small but critical

Downtime reasons come from a fixed dropdown, never free text. A typical plant needs 15–20 codes, set up during onboarding. Examples: Tool Breakage, Tool Change (planned), Material Wait, Machine Breakdown, Setup/Changeover, Quality Hold, Power Cut, No Operator, Planned Maintenance, Meeting/Training. Each code is pre-classified as Recoverable (the plant’s own process failure — fixable) or Structural (unavoidable — planned maintenance, power cuts). This classification is what lets Layer 2 tell the owner which losses they can actually get back.

The Job Order spine — the design decision everything hangs on

The system does not revolve around machines. It revolves around Job Orders. When the supervisor starts a job at shift start, that job order already carries: part number, customer, lot/batch number, machine, operator(s), shift and date. From that moment, every event recorded — every production count, every downtime, every rejection — automatically inherits all of those tags. Nobody types a customer name or a lot number during the shift, ever. The tags travel with the job.

What that gives us for free: traceability

Because every event is stamped with the full job context, the plant gets automotive-grade traceability as a byproduct of daily use — not as a separate module with separate data entry:

Scenario: an OEM customer complains about burrs on lot #4471, three weeks after despatch. Today, answering that means two days of digging through registers. With the job-order spine, the owner asks and gets the answer in seconds: lot #4471 was Job Order 2231, run on CNC-3, Shift A, 14 June, operator Kumar, inspected by Kavitha, 362 good / 9 rejected — 7 of them burrs — and CNC-3 had a tool-breakage event that same morning. Root cause, machine, person, and evidence: all connected, instantly.

Traceability scope at launch: job → part → customer → machine → operator → shift → date. Phase 2 (later): material batch, tool ID, and supplier lot — these require Stores discipline the plant does not have yet, and forcing them at launch would break the 15-minute daily entry budget. They plug into the same spine when the plant is ready.

Multi-stage plants: assembly lines, long throughput, and routings

Single-operation machining plants need only the simple job order above. Assembly plants — where a part passes through several lines over 3–6 days — need one more concept: the routing. A routing is the ordered list of operations a job passes through: Op10 Machining → Op20 Welding → Op30 Assembly → Op40 Testing. Each stage logs quantity completed (and rejected) against the same lot ID, so the system always knows where every lot physically sits.

Lot #4471, 500 pieces, Day 3 of production: 500 completed Op10 (Mon), 430 through Op20 (Tue — 70 in WIP buffer), 380 through Op30, 20 rejected at Op30 welding-distortion, 0 started Op40. Throughput time so far: 2 days 6 hours. The owner sees exactly where the lot is stuck and what the Op30 rejections cost.

The lot identity travels physically as a printed QR code on the route card or bin card — printed paper, zero hardware, zero cost. At each stage the operator scans it (or taps the lot from a short list) and enters the quantity. Throughput time per lot = first-operation start to last-operation completion, computed automatically. Downtime, quality and cost logic all work per operation exactly as they do per machine.

If the plant already runs kanban

Do not replace it — piggyback on it. The kanban card already IS a lot identity moving with material. Print the QR on the kanban card; a card movement between stages becomes a stage-completion event in the system. The plant's physical pull system stays untouched; FactoryManager AI simply reads what the cards were already doing. This is also the sales answer when a plant says 'we already have kanban': good — then you are one printed QR away from digital traceability.

If the plant has no lot discipline at all (starting from scratch)

Start coarse: the day-batch is the lot. Everything Line 2 produces on 14 June is lot L2-140626 — one rule, implementable on day one, and already better traceability than most tier-2 plants have. When a customer complaint arrives, the trace narrows to one line and one day instead of 'somewhere last month'. As discipline grows: day-batch → shift-batch → true job-order lots. The system supports all three from day one; the plant climbs the ladder at its own pace, and the Digital Readiness Score (Section 4.1) tells us which rung they start on.

How 'live' works without sensors

Two speeds. Downtime is event-driven — it is logged the moment it happens, so the rupee ticker on a stopped machine is genuinely live. Production counts are hourly — the dashboard refreshes every time someone saves an entry. This is the doctrine: owners do not need second-by-second data to make better decisions; they need timely-enough data that drives action today, instead of a padded register discovered next week. Hourly is timely enough. Sensors later make it second-by-second — nothing else changes.

Layer 2 — Factory Brain (the money and decision layer)

What it is

Factory Brain takes the raw events from Layer 1 and does two things: it converts every event into rupees, and it ranks everything by money so the plant always knows the single most valuable thing to fix. This layer is why owners pay. Nobody buys 'OEE tracking' — they buy 'you lost ₹9,380 yesterday and here is how to stop it'.

The cost calculation — full worked example

During onboarding, five numbers are entered once per machine/part combination (Section 4 lists them all). Using those numbers, the 47-minute tool breakage on CNC-3 (running Part AK-447) is costed automatically the moment the supervisor logs it:

Cost component	Calculation	Amount
Machine cost (idle machine still costs money)	47/60 hr × ₹420/hr	₹329
Labour cost (operator standing idle)	47/60 hr × ₹180/hr	₹141
Overhead (factory costs keep running)	47/60 hr × ₹210/hr	₹165
Lost contribution (parts NOT made in 47 min)	(47 ÷ 2.4 min cycle) = 19 parts × ₹38 margin	₹2,655
TOTAL cost of this one event		₹3,290

Notice that the biggest loss is not the machine or labour — it's the 19 parts that were never made. This is the number owners have never seen before, and it is why the product sells itself in a demo.

The ranking — what to fix first

At the end of each shift, the Brain produces a ranked loss table. Not ranked by frequency, not by minutes — by rupees, and split into Recoverable vs Structural:

#	Machine	Reason	Duration	Cost	Type
1	CNC-3	Tool breakage	47 min	₹3,290	Recoverable
2	Line 2	Material wait	31 min	₹2,170	Recoverable
3	VMC-1	Minor adjustments	22 min	₹1,540	Recoverable
S	All lines	Planned maintenance	34 min	₹2,380	Structural
	TOTAL		134 min	₹9,380	

The AI Shift Coach — the Brain speaking

Every morning at 5:45 AM, before the shift starts, the system reads the plant's accumulated history (it becomes genuinely useful after 4–6 weeks of data) and sends the incoming supervisor a briefing with exactly three priorities. Each one names a machine, cites the plant's own history, gives a concrete action, a person, a deadline, and a rupee value. A real example of what it generates:

> **PRIORITY 1 — CNC-3: High tool-breakage risk today.** CNC-3 averages a tool breakage every 4.2 days. Today is Day 4 since the last one. 73% of breakages on this machine happen in the first 90 minutes of the morning shift. Yesterday's event cost ₹3,290. ACTION before 7:15 AM: inspect and replace the insert on Station 3. Assign to Suresh K. (Maintenance). Prevention value: ~₹3,200.

In the first weeks — before enough history exists — the Coach runs on simple rules (e.g. 'the same reason code appeared 3 times in 5 shifts with no action logged → flag it'). It gets smarter as data accumulates. It never gives generic advice; if it has nothing specific to say, it says less.

Escalation — the Brain acting in real time

The Brain doesn't wait for shift end. When a downtime event stays open: at 5 minutes the supervisor gets a push notification; at 15 minutes a WhatsApp message goes automatically to Maintenance (for breakdowns) or Stores (for material waits) — the system picks the recipient from the reason code; at 30 minutes the Plant Head gets an alert with the running cost. Every escalation is timestamped. Nobody can later say they didn't know.

Layer 3 — Factory Copilot (the conversation layer)

What it is

Once Layers 1–2 are running, all the plant's data exists in one place. The Copilot is a WhatsApp interface on top of it: anyone in the plant can ask a question in plain language (English or Tamil) and get an answer computed from their own plant's data. No dashboards to learn, no reports to dig through — just ask.

Example conversations

Who asks	Question (via WhatsApp)	What the Copilot answers
Owner	"How much did we lose this week?"	"₹41,200 recoverable loss this week, down ₹6,800 from last week. Biggest: material waits on Line 2 (₹9,400)."
Supervisor	"Why is my line behind?"	"Line 2 lost 41 minutes to material wait this morning (₹2,870). Stores was alerted at 9:15, unresolved. Escalate to Plant Head?"
Maintenance	"Which machine should I look at first?"	"CNC-3 — 4 tool breakages in 18 days, ₹12,900 total. Next highest: VMC-1 Monday-morning pattern."

Who asks	Question (via WhatsApp)	What the Copilot answers
Quality	"Which defect is increasing?"	"Burr on Part AK-447: 0.8% → 2.1% over 3 weeks. All from CNC-3, all after tool changes."

Available from Day 1 — with honest limits, enforced by design

The Copilot ships from day one, but a Copilot with three days of history that confidently answers pattern questions will be confidently wrong — and one wrong answer to an owner kills trust in every number. So its scope is tiered and the system enforces the tiers; this is not left to a sales disclaimer:

- **Day 1: factual answers only.** 'What happened today?' 'How much downtime this week?' 'Which machine lost the most yesterday?' These are correct even with a week of data.
- **Unlocked with history: pattern answers.** 'Why does CNC-3 keep failing?' 'What usually fixes this?' The Copilot answers these only once enough events exist to support them — and until then it says so: 'I've seen 2 events like this; I'll give you pattern analysis once I've seen more.'

Important: the Copilot is not a new intelligence — it is the same Brain data, delivered through conversation instead of screens. It reuses the same WhatsApp channel already set up for escalations. This is why we apply for the WhatsApp Business API (WATI) on Day 1: approval takes 3–5 weeks in India, and both escalations and the Copilot depend on it.

Layer 4 — Factory Memory (the knowledge layer)

What it is

Every plant slowly loses its knowledge. An experienced engineer leaves, and with him go the tool settings, the machine quirks, the supplier issues, the root causes of old problems. Factory Memory captures all of it — because every downtime event, every fix logged by maintenance, every defect and its resolution is already flowing through Layers 1–2 — and makes it searchable forever.

Example scenario

A new quality engineer, 14 months from now, spots an unusual rust mark on incoming brackets. She asks the Copilot: "Have we ever had rust spots on brackets before?" Answer: "Yes — July 2026, Machine 6, Supplier ABC batch #4471. Root cause: humidity during monsoon storage. Fix: desiccant packs added to incoming pallets. Logged by Ramesh P." A problem that would have taken two days of asking around is solved in thirty seconds.

The knowledge-capture prompt — how Memory gets fed

Memory records from day one, and one lightweight mechanism feeds it deliberately: when a significant Action is closed (Section 5), the system asks the person who fixed it four quick questions. These four questions are the standard Factory Memory capture set:

1. What fixed it? (the countermeasure that worked)
2. What did you replace or change? (part / setting / material / method)
3. Attach a photo (optional).
4. What should someone check first next time? (the fast diagnostic for the future)

Sixty seconds, once per significant fix. Fifty such closures later, the plant owns a private repair playbook that survives any resignation — and this accumulated, plant-specific knowledge is one of the product's deepest moats.

This layer otherwise requires no new data entry — it is a byproduct of using Layers 1–2 over months. That is also why it cannot be rushed: Memory with three weeks of history is worthless; Memory with two years of history is nearly impossible for a competitor to copy, because they would need the plant's two years of data to match it. This is the long-term moat.

Layer 5 — Factory Plugins (built in parallel)

Three plugins are developed in parallel with the core product, not deferred: Compliance/Audit AI, Kaizen AI, and Skill Matrix AI. They are separate build streams, but they are never separate products — no separate logins, no separate data entry. Each one feeds on the same Factory Core data the plant is already capturing daily and writes back into the same Job Order spine, Actions engine and Factory Memory.

> **The one rule that governs every plugin:** it must feed on existing Core data, not demand new daily data-entry burden. If a plugin needs the plant to do more daily work, it is designed wrong. The 15-minute daily entry budget (Section 4.3) is sacred and applies across the whole platform, plugins included.

Compliance / Audit AI — scope

Generates audit evidence (ISO 9001, IATF 16949, ZED, customer-specific / OEM formats) automatically from data the plant already captures daily, and scores live audit readiness against the relevant clauses. A downtime log with corrective actions IS audit evidence — the plugin formats it against the right clause and tracks CAPA to closure through the same Actions engine. Cross-standard mapping means one captured fact satisfies multiple standards at once.

Kaizen AI — scope

Operators submit improvement ideas (eventually by voice, in Tamil); the system converts each into a structured cost-saving proposal, scores it, routes it into the Actions engine for implementation, and tracks the realised saving through the same Recovered-Profit accounting the core product already uses. Reliable pattern detection needs roughly 60–90 days of core data first, so early Kaizen runs on operator submissions and simple rules, deepening as data accumulates.

Skill Matrix AI — scope

Tracks which operator is trained on which machine/operation, and connects that matrix directly to downtime data — 'No Operator' events point at exactly where a skill gap is costing money, and the plugin ranks training needs by the rupee loss they would remove. It reuses the operator list and event stream already in Core; no new registers.

Build rule for all three plugins

- One shared data model. Plugins read Core; they never fork the data.
- One Actions engine. Compliance CAPAs, Kaizen implementations and Skill-gap fixes are all Actions (Section 5) — same lifecycle, same PDCA, same closure prompt into Factory Memory.
- One rupee ledger. Every plugin outcome is expressed in the same Recovered-Profit terms as the core, so the owner's money screen stays the single source of truth.

Layer 6 — Industry Benchmark (the network layer)

Once 40–50 plants run the system, we can show each plant how it compares to similar plants (fully anonymised, with permission): "Plants like yours average 71% OEE — you are at 64%. Top performers change tools every 240 parts; you change at 310." No individual plant can create this on its own, and it becomes more valuable with every

plant that joins. This begins when the installed base reaches 40–50 plants — it appears in this document only so the team understands where the road leads. Nothing is built for it now.

Section 4 — The Data: Exactly What the System Needs

The entire product runs on two kinds of data: setup data entered once during onboarding, and daily data entered every shift. This section is the complete list. If it's not here, the system doesn't need it.

4.1 Onboarding — upload first, type last (target: half a day, not three)

Onboarding is the single biggest adoption risk in this product. If setup takes three days of manual typing per plant, it does not scale and plants lose patience before they see value. So onboarding is redesigned around one rule: never type what can be uploaded, never demand what can be estimated.

Step 0 — Digital Readiness Questionnaire (before we ever visit)

A short questionnaire (25–30 questions) the owner answers on their phone before the first visit: Do you use ERP, Tally, or Excel? Paper production sheets? Any OEE tracking today? Quality records? Maintenance registers? Barcodes? The system returns a Digital Readiness Score — e.g. '41/100. You can start immediately with manual capture. Estimated onboarding: half a day. Priority gaps: downtime logging, traceability.' This qualifies the plant before we spend a day there, tailors the onboarding, and is itself a powerful sales artefact.

Step 1 — Upload the masters they already have

Before the visit, we ask for whatever exists: part master, machine list, employee list, route cards / cycle-time sheets, production plan — as Excel, Tally exports, or even photographed registers. The system extracts and proposes the configuration: 'Found: 132 part numbers, 19 machines, 41 operators, 3 shifts. Confirm?' A 30–60 minute verification replaces days of typing. Reality check for the build team: most target plants run Tally plus paper, not ERP — so design for structured Excel templates and tolerant extraction, not ERP integrations (those are Year 2+).

Step 2 — The floor walk (the human part that cannot be uploaded)

The onboarding consultant walks the floor with the production head: verify the machine list against reality, sanity-check cycle times on 3–4 key parts, agree the downtime reason codes with the supervisor who will actually use them, and set the cost parameters with the owner privately (Section 4.2). Then week 1 is habit training, not configuration.

The don't-block rules

- **No cycle times?** Estimate from past production history (parts ÷ hours from any old register) or route cards; refine during week 1 on the floor. Never block.
- **Owner won't share exact margins?** Use Low / Medium / High contribution bands per part family. The loss ranking stays correct; precision comes when trust does.
- **No cost data at all?** Start in Basic costing mode (Section 4.2) with one blended rate per machine. Fifteen minutes with the owner.

The complete onboarding data list

Whether uploaded or entered, this is everything the system needs — nothing else:

Data item	Example	Where it comes from
Machine list — every machine, type, ID	CNC-3 (CNC turning), VMC-1 (milling), Press-2 (100T)	Walk the floor with production head

Data item	Example	Where it comes from
Machine cost rate per hour	CNC-3: ₹420/hr	Depreciation + power + tooling — owner/accountant, once
Labour cost per hour per line	₹180/hr (operator + supervisor share)	HR / wage records
Overhead absorption rate per hour	₹210/hr	Total factory overhead ÷ available machine hours
Part master — every part number produced	Part AK-447 'Bracket LH'	Production records
Standard cycle time per part per machine	AK-447 on CNC-3: 2.4 min/part	Time study or existing route cards; validated on the floor
Contribution margin per part	AK-447: ₹110 − ₹72 = ₹38/part	Owner (confidential; can be a range if sensitive)
Shift pattern	2 shifts: 7 AM–3:30 PM, 3:30 PM–12 AM	Production head
Downtime reason codes (15–20), each tagged R/S	Tool Breakage (R), Material Wait (R), Power Cut (S)…	Drafted from experience, tuned with supervisor
Defect codes (10–15) per part family	Burr, Undersize, Rust Spot, Tool Mark…	Quality head
People and roles — who gets which alerts	Suresh K. = Maintenance, Ramesh P. = Stores, owner	Owner
Daily production target per line	Line 2: 380 parts/shift	Production plan

This onboarding consultation IS the sales wedge. Even before any software exists, walking a plant through this exercise — 'let's put a rupee figure on your downtime for one week, on paper' — proves the value, builds the owner's trust, and produces the exact dataset the software needs on day one.

4.2 The three costing modes — rupee figures without an accountant

Cost setup was the scariest part of onboarding, so it is tiered. The system does not need accounting-grade cost data; it needs operational costing — numbers good enough to rank losses correctly. A ±20% cost estimate still puts the biggest loss at the top of the list, and the ranking is what drives action, not the third decimal.

Mode	What is entered	Time to set up	Who it suits
BASIC	One blended hourly rate per machine + one contribution band (L/M/H) per part family	15 minutes with owner	Every plant on day one. Nobody is blocked.
STANDARD	Machine rate + labour rate + overhead rate + exact contribution per part	Half a day with owner + accountant	Plants ready for precise figures, after 4–8 weeks of trust
ADVANCED	Activity-based costing, ERP-linked actuals	Year 2+, not at launch	Mid-size plants; explicitly out of MVP scope

How to derive a Basic blended rate in 15 minutes

Blended machine rate per hour = (monthly depreciation share + monthly power share + operator wages + maintenance allowance + factory-overhead share) ÷ monthly available machine hours. Worked quickly with an owner: a CNC bought at ₹35L depreciating over 10 years (≈₹29K/month) + ₹18K power + ₹28K operator + ₹10K

maintenance + ₹40K overhead share ≈ ₹1.25L/month ÷ 400 available hours ≈ ₹310/hr, plus contribution band per part. Most CNC/VMC machines land between ₹600–900/hr once contribution is included. The owner nods along because every number is theirs.

> **Rule for the build team:** the cost engine must run identically in all three modes — same formulas, different granularity of inputs. A plant upgrades from Basic to Standard by filling in more fields, never by migrating.

4.3 Daily data — entered every shift (by plant staff)

Data item	Entered by	Frequency	Fields
Shift start record	Supervisor	Once per shift	Shift, date, line/machine, job order, part number, operator name(s)
Production count	Supervisor or operator	Hourly, or per batch	Machine, count since last entry, timestamp (auto)
Downtime event	Supervisor	Per event	Machine, reason code, start (auto), end (auto on 'Running'), optional note
Quality record	Quality inspector	Per batch, 2–4×/shift	Part number, good count, reject count, defect code(s), disposition
Action item update	Whoever owns the action	As done	Action status: Done / Delayed + reason
Shift closure	Supervisor	Once per shift	Confirmation; system blocks closure if anything above is missing

Total daily data-entry burden for a supervisor: roughly 10–15 minutes spread across an 8-hour shift. This number matters — if the burden creeps above that, adoption dies. Every screen must be designed against this budget.

4.4 What the system produces from that data (outputs)

Output	Goes to	When
Live line status: running/stopped, count vs target, cost ticker for open downtime	Shopfloor screen / supervisor tablet	Real time
Escalation alerts (5 / 15 / 30-min ladder)	Supervisor → Maintenance/Stores → Plant Head (push + WhatsApp)	Real time
End-of-shift cost report: ranked losses in ₹, recoverable vs structural	Owner's phone (WhatsApp/app)	At shift close
Morning-meeting pack: locked numbers, top 3 downtime + top 3 quality by cost, actions gate	Meeting screen / any device	8 AM daily
AI Shift Coach briefing: 3 prioritised, costed, named actions	Incoming supervisor's phone	5:45 AM daily
Monthly owner report: total recoverable loss, trend, annualised projection, top recurring problems	Owner	Monthly
Copilot answers to plain-language questions	Anyone, via WhatsApp	On demand

Section 5 — The Actions & PDCA Engine

Everything before this section makes problems visible. This section makes them SOLVED — and proves it in rupees. Without it, the product is an expensive mirror: the plant knows its problems but not whether they ever got fixed. With it, the product closes the improvement loop, which is what owners actually renew for.

5.1 What an Action is

An Action is a tracked commitment to fix something, born from an event. Every Action carries: what the problem is (linked to the underlying events and their cost), who owns it (a named person), a deadline, a status, the root cause once known, the countermeasure taken, optional photo evidence, and — after closure — the measured savings. An Action is never free text in a meeting minute; it is a living object the system tracks until it is closed and verified.

5.2 When Actions are created — the trigger rules

Not every downtime becomes an Action. A 4-minute tool adjustment does not need a PDCA cycle, and a system that spawns forty open actions a week dies of action fatigue within a month. Actions are created three ways:

Trigger	Rule (configurable per plant)	Example
Big single event	Any event costing more than ₹2,500 or lasting more than 30 minutes auto-creates an Action	47-min tool breakage, ₹3,290 → Action created automatically at event closure
Recurring pattern	Same reason code on the same machine/line 3+ times in a rolling 7 shifts → the AI creates one Action covering the pattern	Material wait on Line 2, 3rd occurrence → Action: 'Eliminate short-staging on Job #4471', owner: Stores
Manual	Anyone can raise an Action from any event or observation (this is also where Kaizen ideas enter)	Supervisor raises Action from a near-miss he noticed

Everything below the thresholds stays a logged event — visible, costed, counted in KPIs, but not generating work. The thresholds are set during onboarding and tightened as the plant matures.

5.3 The morning meeting routes issues to PDCA/RCA by severity — not all issues

This is the key behaviour the build team must implement precisely. The morning meeting does NOT ask the team to run a root-cause analysis on every issue from yesterday. That is how RCA dies — nobody does five 5-Whys before 8 AM. Instead, the system triages, and only pushes issues into PDCA / root-cause analysis when the lost time or severity crosses the threshold:

1. **The meeting screen shows yesterday's ranked losses.** Everything is visible and costed, but not everything demands work.
2. **The system decides which issues escalate to PDCA/RCA, by severity.** An issue is routed into a root-cause / PDCA cycle only if it meets a severity rule — e.g. lost time or cost above the Action threshold (5.2), OR a safety/quality-escape severity flag, OR a recurring pattern (3+ in the rolling window). Small, one-off losses stay logged and costed; they are not sent to RCA.
3. **For the issues that do escalate, the meeting takes the team straight into the PDCA/RCA flow** for that specific issue — root cause known → record it and the countermeasure; root cause unknown → assign an RCA with a pre-filled 5-Why template and a deadline. The team spends its RCA effort only where the money or severity justifies it.

4. **The meeting cannot advance to Today's Plan** while yesterday's escalated Actions sit unaddressed (each must be Done or Delayed-with-reason). Discipline and the improvement loop become the same mechanism — but only for the issues that earned a place in it.

On closure, the system asks the four Factory Memory questions. When an escalated Action is closed, the person who fixed it is asked the standard four capture questions (Section 3, Layer 4): (1) What fixed it? (2) What did you replace or change? (3) Attach a photo (optional). (4) What should someone check first next time? The answers are recorded into Factory Memory — so the same problem is diagnosed in seconds the next time, by anyone.

5.4 The Action lifecycle IS the PDCA cycle

Plants know PDCA — Plan, Do, Check, Act. The Action lifecycle maps onto it exactly, so nothing feels foreign:

PDCA	Action state	What happens
PLAN	Open → Root cause → Countermeasure planned	Problem defined with its ₹ cost attached; root cause established (5.5); countermeasure, owner and deadline set
DO	In progress	The fix is executed; photo/evidence can be attached
CHECK	Closed – monitoring	The system watches the linked reason code for 2–4 weeks: did the events actually stop?
ACT	Verified (or Reopened)	If the loss rate stayed down → Verified, savings computed and credited (5.6). If events recur → automatically Reopened — a fix that didn't hold is not a fix

5.5 Root cause — reason codes are not root causes

The reason code ('Machine Breakdown') is logged in 20 seconds during the shift — it says WHAT happened. The root cause ('bearing seized — wrong grease grade used at last PM') says WHY, and it is established later, at Action level, never on the shopfloor mid-shift. The flow:

- **Root cause already known:** whoever closes the Action writes it directly — one field, plus the countermeasure. Most tool and material issues land here.
- **Root cause unknown:** the Action owner assigns an RCA to the relevant team — the system sends them an email/WhatsApp with a simple 5-Why template pre-filled with the event history (dates, frequency, cost so far) and a deadline. The Action stays open, visibly, until the RCA returns.
- **AI assist:** when enough history exists, the AI pre-suggests likely root causes from the plant's own past ('the last 3 breakdowns on this machine were bearing-related; last PM was 47 days ago') — a suggestion to accept or reject, never an automatic verdict.

Root-cause entries accumulate into Factory Memory (Layer 4) — which is exactly how the 'have we seen this before?' answers get their substance.

5.6 Measured savings — the number that renews contracts

Recovered Profit is the product's north-star metric, and it must be measured conservatively or it becomes a dispute. The rule:

Recovered Profit for an Action = (baseline weekly loss for that reason/machine, averaged over the 4 weeks before the Action) minus (actual weekly loss after closure), summed weekly — counted only after the fix has held for 2 consecutive weeks, and never below zero.

Example: material waits on Line 2 were costing ₹4,400/week (4-week baseline). Action closed; the following weeks run at ₹1,100/week. Recovered Profit accrues at ₹3,300/week from week 3 onwards — ₹1.7 lakh annualised, credited to that Action and to its owner. The owner's monthly report leads with: total ₹ recovered this month, cumulative ₹ recovered vs annual fee (the running ROI), and the top open opportunities. Nobody cancels a system whose home screen shows 19× ROI.

Section 6 — A Day in the Life (Full Worked Scenario)

Fictional plant: Kumaran Auto Components, Hosur. 18 machines, 2 shifts, tier-2 supplier to a major OEM's tier-1. Six weeks into using FactoryManager AI. Follow one full day:

5:45 AM — The AI Coach speaks first

Supervisor Murugan's phone buzzes before he leaves home. Three priorities: (1) CNC-3 tool-breakage risk — inspect insert before 7:15, assign Suresh, value ₹3,200. (2) Job #4471 material has been short-staged 3 times in 5 shifts — confirm with Ramesh in Stores that full quantity is at Line 2 before 8:00. (3) VMC-1 has dipped below 58% OEE every Monday for 4 weeks — check coolant concentration before starting.

7:00 AM — Shift start

Murugan opens the tablet, selects Shift A, confirms the job orders per line (part numbers, operators pre-filled from yesterday's plan, edits one operator swap), taps Start. Under a minute. Suresh has already changed the CNC-3 insert — Priority 1 done, logged with one tap.

8:00 AM — Morning meeting, changed forever

The meeting screen shows yesterday's locked numbers — nobody argues about them because nobody can edit them. Top loss: ₹3,290 tool breakage. Second: ₹2,170 material wait, third occurrence. The system routes only these two into PDCA/RCA — they cleared the severity threshold; the six small sub-₹500 losses stay logged, not sent to RCA. The system will not show 'Today's Plan' until yesterday's escalated actions are marked Done or Delayed-with-reason. The Stores short-staging finally gets a named owner and a deadline, because it has now cost ₹6,510 across three events and everyone can see the number. Meeting length: 14 minutes.

10:40 AM — A downtime event, live

Press-2 stops — material wait, again. Murugan logs it in 20 seconds. A rupee counter starts climbing on the line screen. At 5 minutes, his phone buzzes. At 15 minutes, Ramesh in Stores gets an automatic WhatsApp: 'Press-2 waiting on material for Job #4482 — 15 min, ₹1,050 and counting.' Material arrives at minute 22. Event closed: ₹1,540. Ramesh knows the owner sees the same number at shift end.

2:30 PM — Quality entry

Inspector Kavitha completes the AK-447 batch check: 362 good, 9 rejected — 7 burr, 2 undersize. She logs it in a minute. The system quietly cross-checks: Murugan's hourly counts sum to 374 for that batch; 362 + 9 = 371. Three parts unaccounted — within tolerance, no flag. If the gap were 25 parts, the owner would get an automatic reconciliation alert.

3:25 PM — Shift closure

Murugan taps Close Shift. The system blocks: one downtime event from 1:10 PM has no reason code. He selects 'Setup/Changeover'. Now it closes. The data locks permanently.

3:27 PM — An Action is born

Because the Press-2 material wait was the third same-reason event in the rolling window, it clears the severity rule and the system creates one pattern Action at shift close: 'Eliminate short-staging on Job #4482 line feeds — 3 events, ₹4,760 to date. Owner: Ramesh P. (Stores). Deadline: Friday. Root cause: pending — RCA assigned to Stores with 5-Why template.' It will sit on the Actions Board and in tomorrow's morning meeting until closed — and after closure the system will watch for two weeks to verify the waits actually stopped before crediting the savings, then ask the four Factory Memory questions.

3:35 PM — The owner’s phone

Owner Mr. Kumaran, at a customer meeting in Chennai, gets the shift report: ‘Shift A: 2,140 parts (target 2,280). Recoverable loss ₹7,890. Top: material wait Press-2 ₹1,540; tool change VMC-2 ₹1,310… Month so far: ₹1.02L recoverable. Annualised: ₹28.4L.’ He reads it in 90 seconds. For the first time in 22 years of running the plant, he knows what today actually cost him — while it is still today.

Section 7 — The Screens (Role-Based, Money-First)

Functional description only — visual design and layout are the build team's craft. Ten screens make up the launch product. Nothing else is needed for MVP.

The most important design decision: home screens are role-based, and the owner's home is money, not machines. An owner opening the app sees four numbers and nothing else: Recovered Profit yesterday, Lost Profit yesterday, Actions completed, running ROI. Every number is a door — tap Lost Profit → ranked losses → machine → events → the Action fixing it, its root cause and photo. The supervisor's home stays the machine grid, because he is running a shift, not reading a P&L. Same data, different doors.

#	Screen	User	What it must do
1	Shift Start	Supervisor	Pick shift → confirm lines/jobs/parts/operators (pre-filled, editable) → Start. Under 60 seconds.
2	Machine Grid (home)	Supervisor	One tile per machine: green running / red stopped, count vs target, live cost ticker on any open downtime. Tap to act.
3	Downtime Log	Supervisor	Tap 'Stopped' → reason dropdown (biggest buttons for commonest reasons) → optional note. Tap 'Running' to close. Two taps min.
4	Production Entry	Supervisor/Operator	Enter count since last entry per machine. Hourly reminder nudge. Number pad, 15 seconds.
5	Quality Entry	Inspector	Part number → good count → reject count → defect codes → disposition. Independent login from supervisor.
6	Shift Closure	Supervisor	Checklist of blocking items (missing reasons/counts, open escalations). Resolve or request owner override (logged). Close → lock.
7	Morning Meeting Mode	Whole team (big screen)	Locked numbers, top 3 downtime + top 3 quality by ₹, actions gate before Today's Plan. Readable from 5 metres.
8	Owner Home (Money Screen)	Owner (phone)	Four numbers: Recovered ₹, Lost ₹, Actions done, running ROI — each drills to losses → machines → events → Actions. Daily report via WhatsApp + in-app.
9	Actions Board	Supervisor / action owners	Open, In-progress, Monitoring, Verified. Each Action: linked events + ₹, owner, deadline, root cause, countermeasure, photo. Overdue items surface in Morning Meeting Mode.
10	RCA & Closure Prompt	Whoever closes an Action	Root-cause field (or assign RCA with 5-Why template + deadline), countermeasure, photo, and the four Factory Memory questions (60 seconds).

Two non-screen deliverables complete the MVP: the escalation engine (the 5/15/30-minute WhatsApp ladder) and the AI Shift Coach briefing (rule-based at first, delivered 5:45 AM daily). Everything must work offline on the tablet and sync when internet returns — Tamil Nadu plant Wi-Fi cannot be trusted. Language: English primary; Tamil available via an info icon on labels. Every screen designed for thick fingers, poor light, and a hurry.

Section 8 — KPIs and the Formula Reference

Once shift data flows hourly, a large family of KPIs becomes computable — OEE, productivity, downtime rates, quality rates and more. Two iron rules govern all of them:

- Every KPI is derived from data already being entered. If a proposed KPI would require new daily data entry, it is rejected. The 15-minute entry budget is sacred.
- No KPI zoo. Launch with roughly a dozen KPIs across all roles — one executive page, drill-down per role. A new KPI is added only when a real user asks for it twice.

OEE, explained once

OEE (Overall Equipment Effectiveness) = Availability × Performance × Quality. Availability: of the planned shift time, how much did the machine actually run? Performance: while running, how close to the standard cycle time was it? Quality: of the parts made, how many were good? Example: 85% × 90% × 97% = 74% OEE. Every input already exists in the daily data — OEE costs the plant nothing extra.

The role-based KPI set at launch

Role	Their page shows	Why these
Owner (executive)	Recoverable loss ₹ (day/month/trend), ₹ recovered vs last month, Top 5 losses, quality PPM, delivery (later)	Money first. The owner should never need to understand OEE to use this product.
Supervisor	OEE per line, production vs target, downtime minutes by reason, open actions	What they control this shift, nothing they don't
Quality	PPM, first-pass yield, top 3 defects with trend, rejects by machine	Points them at the machine causing tomorrow's complaint
Maintenance	Breakdown count per machine, MTTR, MTBF	Turns 'CNC-3 keeps breaking' into a number that ranks their day

The north-star KPI is Recovered Profit (Section 5.6) — not OEE, not downtime, not PPM. Those are diagnostic; Recovered Profit is why the owner renews. Every other KPI exists to feed it. Everything on every page is clickable down to the underlying events.

8.1 The complete formula reference (for the build team)

Every calculation in the product, in one place. Symbols: D = downtime duration (minutes); CT = standard cycle time (min/part); MCR/LCR/OHR = machine/labour/overhead cost rates (₹/hour); CM = contribution margin (₹/part); BF = bottleneck factor (see 8.2).

Cost engine

Calculation	Formula	Notes
Blended machine rate (Basic)	(monthly depreciation + power share + operator wages + maintenance allowance + overhead share) ÷ monthly available machine hours	One rate per machine; 15-min setup
Machine cost rate MCR (Standard)	(depreciation + power + tooling + maintenance) ÷ available hours	Per machine

Calculation	Formula	Notes
Labour cost rate LCR (Standard)	(operator wages + allocated supervisor share) ÷ worked hours	Per line/machine
Overhead rate OHR (Standard)	total factory overhead ÷ total available machine hours	Same rate plant-wide
Contribution margin CM	selling price − direct material cost − variable outwork cost	Per part; L/M/H bands allowed
Downtime event cost	(D÷60) × (MCR + LCR + OHR) + ⌊D÷CT⌋ × CM × BF	Runs live as the cost ticker; ⌊ ⌋ = round down
Scrap cost (rejected, unrecoverable)	material cost + Σ(processing cost of completed operations) + CM × BF	Processing cost per op = (CT÷60) × (MCR+LCR+OHR)
Rework cost	(rework time ÷ 60) × (MCR + LCR + OHR) of the rework station	Reworked parts are NEVER also counted as scrap
Recoverable loss (shift/day/month)	Σ event costs where reason code is tagged Recoverable	Structural losses reported separately, never mixed

8.2 The bottleneck factor — the correction that protects credibility

Lost contribution ('parts never made') is only fully real when the stopped machine constrains the plant's output. If a non-bottleneck machine stops for 47 minutes and the downstream buffer absorbs it, despatch does not drop — and a plant manager who checks will find the claimed ₹2,655 never left the building. One inflated number discredits every number. So:

- BF = 1.0 for the bottleneck machine(s), for any event that stops the whole line, and as the default where unknown
- BF = 0 to 0.5 for non-bottleneck machines with downstream buffer — set per machine during onboarding by whoever walks the floor (this is exactly the Lean judgment the implementation consultant brings)
- Machine/labour/overhead costs always apply in full regardless of BF — an idle machine and idle operator cost money whether or not despatch drops

OEE and production KPIs

KPI	Formula	Data source
Planned production time	shift duration − planned downtime (breaks, planned maintenance, no-order time)	Shift pattern + Structural reason codes
Availability A	(planned time − unplanned downtime) ÷ planned time	Downtime logs (Recoverable codes)
Performance P	(CT × total count) ÷ actual run time	Hourly counts + cycle times; cap at 100%, flag if above
Quality Q	good count ÷ total count	Inspector entries
OEE	A × P × Q	Computed per machine, line, shift, day
Productivity	good parts ÷ man-hours (and per machine-hour)	Counts + shift roster

KPI	Formula	Data source
Downtime %	total downtime ÷ planned time, split R/S	Downtime logs
Schedule adherence	actual output ÷ planned output per job order	Job orders + counts

Quality and maintenance KPIs

KPI	Formula	Notes
PPM (defect rate)	(rejected parts ÷ total parts inspected) × 1,000,000	Per part, machine, and customer
First Pass Yield FPY	parts good first time ÷ parts started	Reworked-then-good parts do NOT count as first-pass
MTBF	total run time ÷ number of breakdowns	Per machine; 'Machine Breakdown' code only
MTTR	total breakdown repair time ÷ number of breakdowns	Repair time = downtime duration of breakdown events
Throughput time (multi-stage)	last-operation completion − first-operation start, per lot	Routing timestamps (Section 3)

Recovered Profit and ROI (the north star)

Calculation	Formula	Rules
Baseline loss rate (per Action)	average weekly ₹ loss for the linked reason+machine over the 4 weeks before the Action	Frozen at Action creation
Recovered Profit (per Action)	Σ over weeks after closure of max(0, baseline − actual weekly loss)	Counted only after 2 consecutive sustained weeks; never negative; stops if Action reopens
Recovered Profit (plant)	Σ across verified Actions	The number on the owner's home screen
Running ROI	cumulative Recovered Profit ÷ cumulative fees paid	Shown monthly; the renewal argument
Annualised projection	current 4-week recoverable loss run-rate × 13	Always labelled as projection, never as fact

> **Build rule:** every formula above must be implemented once, in one place, and every screen/report/Copilot answer must draw from that single implementation. Two screens showing two different OEE numbers for the same shift is a credibility-ending bug.

Section 9 — Build Order for the Team

Feature order only. The logic: each stage must be independently useful, so if we stop at any stage, we still have a sellable product. The three plugins (Section 3, Layer 5) build in parallel on the same Core data — they are separate work streams, not a later stage.

Stage 1 — Core capture + cost engine (the sellable minimum)

- Digital Readiness Questionnaire — a scored form, output as a one-page report (4.1, Step 0). Cheap to build, disproportionate sales value.
- Master-file upload and extraction: structured Excel templates for part master, machine list, employee list, cycle times — with a confirm screen. Strict templates first; tolerant AI parsing is an enhancement, not a blocker.
- Onboarding setup for whatever isn't uploaded: cost rates (Basic + Standard), reason codes, defect codes, people/roles, shift patterns.
- Screens 1–6: shift start, machine grid, downtime log, production entry, quality entry, shift-closure lock.
- The cost engine: every downtime event and rejection costed in ₹ automatically per Section 8, including the bottleneck factor.
- The job-order spine with routing support in the data model (multi-stage UI can follow, but the model supports operations from day one — retrofitting a routing later is a rebuild).
- Owner's end-of-shift report (Screen 8) — WhatsApp delivery.
- Offline capability on the tablet.
- **Data-source abstraction on the capture layer** — so a future sensor feed enters through the same contract as a supervisor entry (Section 2). Cheap now, a rebuild later.

Stage 1 alone is demonstrable and sellable: 'your supervisors enter what happens; you see what it costs, every day, tamper-proof.' Do not add anything else until this works in a real plant.

Stage 2 — Discipline, Actions, and intelligence

- The Actions & PDCA engine (Section 5): trigger rules, severity-based routing to PDCA/RCA, lifecycle, RCA assignment flow, the four-question knowledge-capture prompt, Recovered-Profit calculation. The heart of Stage 2.
- Owner Money Screen home + Actions Board (Screens 8–10).
- Escalation ladder (5/15/30 min, WhatsApp via WATI).
- Morning Meeting Mode (Screen 7) with the severity routing and open-actions gate.
- Quality vs production cross-verification alerts.
- AI Shift Coach — rule-based version.

Stage 3 — Copilot expansion

- Copilot factual tier ships with Stage 1; Stage 3 adds the pattern tier that unlocks with history, plus supervisor and maintenance question sets.
- AI Shift Coach upgraded from rules to AI-generated briefings grounded in 30-day history.

Parallel stream — the three plugins

- Compliance/Audit AI, Kaizen AI, Skill Matrix AI (Section 3, Layer 5) build in parallel on the same Core data model, Actions engine and Recovered-Profit ledger — never as separate products.

Explicitly NOT being built now

- Any hardware, sensors, or firmware (the data layer is designed for it; the devices themselves are the hardware phase).
- Benchmarking, multi-plant dashboards, consultant portals.
- Industry-specific templates (food, foundry, electronics, etc.) — the beachhead is Tamil Nadu auto-ancillary.
- ERP / SAP / Tally integrations and Advanced (ABC) costing — Year 2+; launch takes file uploads.
- Material batch / tool / supplier traceability — Phase 2 of the job-order spine, after Stores discipline exists.
- Tamil voice logging — deliberately deferred until the typed flows are proven; targeted alongside the plugin phase.
- Multi-plant consultant dashboard — not built now, but keep every record plant-scoped from day one so a 100-plant rollup is a screen later, not a migration.

Immediate non-build action: apply for the WhatsApp Business API (WATI) now. Approval takes 3–5 weeks in India and both the escalation ladder and the Copilot depend on it.

Section 10 — Implementation: Transformation, Not Software Rollout

Plants do not experience this as a software installation; they experience a structured improvement programme with the software arriving inside it. That framing matters: MSME owners fear ‘big software projects’ and trust structured improvement programmes. The build team should understand the human wrapper the software lands in, because it shapes what the screens must support.

The onboarding consultant and support roles

The onboarding consultant runs the setup exercise from Section 4.1 (readiness score, master uploads, floor walk, cost setup, reason codes) and trains the plant team in week 1. The core promise is validated on paper first — a hand-built shift-cost report showing the owner ‘here is what last week actually cost you, machine by machine, in rupees’ — before the app arrives. This validates the reaction to the ₹ number, produces real sample data to build and test against, and costs zero development time.

> **For the build team:** the paper-first exercise is the demo your screens should be designed around — real reason codes, real part numbers, real cost rates from real plants. Build to that reality, not to a generic template.

The no-blame rule (build it into the product)

A 90-day no-blame rule is announced out loud by the owner at kickoff: for 90 days, this data is never used against any individual. The product must reinforce it — reports target reasons, machines and processes, never individuals, in every default view. This is a design constraint, not just a policy.

Section 11 — Why Owners Buy, Why They Say No, and the Moat

Why they buy

Owners do not buy DMS, dashboards, OEE, or AI. They buy: profit recovered, OEM approvals kept, more output from the same machines, and less dependence on any one person's memory. The demo is always the same three beats: point at a machine → show what it lost in rupees → show the action that stops it. Machine → Money → Action. Nothing else in the first ten minutes.

Why they say no — and the answer to each

Objection	The honest answer
"My ERP / Tally already does this."	ERP records what happened for accounts. This decides what to fix, today, in rupees. Show one shift cost report next to their ERP screen — the difference explains itself.
"My supervisors won't enter data."	They already write it — on paper, padded. We make it digital, faster than paper, and impossible to pad after shift close. Total burden: 15 minutes per shift.
"Too much work for my people."	15 minutes a shift against ₹2–4 lakh a month of visible recoverable loss. We show the exchange rate in their own numbers by week 2.
"I already know my problems."	You know what goes wrong. You do not know what each problem costs, or which is worth fixing first. Run one week and compare your ranking with the data's.
"Too expensive."	First plants: free pilot. Thereafter: if the first month does not identify recoverable losses exceeding the annual fee, do not continue. The risk sits with us.
"What if my data leaks to a competitor?"	Your data is yours, siloed per plant, never shared. Benchmarking (much later) is opt-in and anonymised. Put it in writing in the agreement.

Why a large competitor can't simply copy this in 12 months

They can build the screens. They cannot easily build these five things, and every strategic decision should consciously deepen them:

- **The cost engine with Lean judgment inside.** Bottleneck factors, recoverable-vs-structural classification, operational costing that survives a plant manager's scrutiny — this comes from 15 years on shopfloors, not from a product team's user interviews.
- **The implementation methodology.** The structured transformation, the no-blame rule, the paper-first validation, reason codes tuned per plant — global tools ship software; we ship behaviour change. This is also why the India MSME segment is unattractive to them: it needs feet on floors, in Tamil.
- **Factory Memory per plant.** Two years of a plant's own root causes and fixes cannot be copied at any price — switching away means abandoning it.
- **Recovered Profit accounting.** A conservative, baseline-verified savings ledger the owner trusts is a renewal machine competitors can't shortcut — it takes months of honest measurement per plant to exist.
- **Benchmark data (later).** At 40–50 plants, the anonymised cross-plant dataset becomes something no new entrant can offer on day one — and it compounds.

The honest corollary: none of these moats exists yet. They are built by winning the beachhead — which is why depth in Tamil Nadu auto-ancillary beats breadth in anything else.

Section 12 — Risk Register: What Could Go Wrong

These are the failure modes that kill products like this in real plants — most have nothing to do with software quality. Each has a mitigation designed in from day one.

#	Risk	What we do about it
1	Data-entry fatigue in weeks 3–6	Shift Closure Lock makes gaps impossible to ignore; morning meeting depends on the data daily; Coach gives supervisors personal value. Watch: % of shifts closed without override.
2	Owner stops reading the report	Report opens are tracked; weekly 15-min review call in weeks 4–8; if opens drop, intervene before the plant decays.
3	Garbage cost parameters	Rates validated personally in onboarding; sanity-check total computed loss against plant revenue; conservative defaults; Basic-mode bands where uncertain.
4	Data used to punish people	90-day no-blame rule announced by the owner out loud at kickoff. The system targets reasons and processes, not names, in every report.
5	Supervisor gaming / under-reporting	Quality vs production cross-verification; count reconciliation alerts; occasional owner spot-check against despatch numbers.
6	WATI/WhatsApp approval delay	Apply on company Day 1; SMS fallback for the escalation ladder in the interim.
7	Tablet/connectivity failures	Offline-first app, sync on reconnect; one rugged tablet per line; conflict rules defined by build team before pilot.
8	Scope creep on the build	The NOT-building list in Section 9 is the contract. Build briefs stay at single-screen level. Anything new goes to a parking lot, reviewed monthly.
9	Savings plateau → churn	The Coach must always surface the next loss; monthly report shows cumulative ₹ recovered vs annual fee — the running ROI is the retention argument.
10	Action fatigue	Trigger thresholds keep Action volume low; severity routing sends only material issues to RCA; recurring events roll into ONE pattern Action; overdue Actions surface in the morning meeting.
11	Savings overclaiming	Baseline frozen at Action creation; 2-week sustain rule; never negative, never counted twice; bottleneck factor prevents phantom lost-contribution claims.

Section 13 — Hardware Upgrade Path and Long-Term Roadmap

Hardware is part of the product from the beginning — the data layer is built for it now. What is deferred is the device work itself: which IoT device is chosen for which class of machine is decided later, plant by plant, once the software is trusted. At that point the natural friction becomes manual entry itself: 'can't this count automatically?' That is the moment hardware sells itself — as an upgrade to a happy customer.

The upgrade swaps manual production entry and manual downtime start/stop for sensor-detected equivalents (e.g. CT-clamp power sensing, controller taps or pulse counters — chosen per machine class, roughly ₹5,500 per machine installed for the common case). Everything above the data layer — cost engine, escalations, morning meetings, AI Coach, Copilot, Memory, plugins — continues unchanged, because none of it ever cared where the data came from. That was the point of designing it this way.

Hardware also opens the premium tier: tamper-proof counts, second-by-second downtime detection, and eventually predictive signals (a machine's power signature drifting before it fails). None of this is designed further here; it exists so the team builds the data layer knowing that one day a sensor feed will plug into the same slot a supervisor's thumb occupies today.

The long-term expansion map (customer lifetime view)

Each plant relationship is designed to expand, not just renew. The sequence a single customer travels: DMS core + Actions (this document) with the three plugins available in parallel → deeper plugin adoption (Compliance/Audit pulled by OEM audit pressure, Kaizen, Skill Matrix, Maintenance) → Benchmarking once the network exists, and a Consultant Edition for multi-plant advisors → Sensors: the hardware upgrade converts manual capture to automatic on the same platform → predictive maintenance from sensor signatures, and eventually plant-level digital twins for the largest customers. Every step sells to an existing, referenceable customer — the platform never has to be re-sold, only extended.

Section 14 — Operational Design: Deployment Questions Answered

This section answers the five questions every implementation and every developer hits first: where the system actually starts in a plant, how multi-stage processes are physically monitored, what happens when a job changes mid-run, how costs are collected at scale, and what the AI Coach needs. Nothing here changes product scope — it operationalises the spec above.

14.1 Where it starts — the minimum company data, and deliberately no more

The system starts from two uploads, not from configuration screens: the machine/station list and the part master. Everything else attaches to those two. Daily job orders are then created from the plant's production plan each morning (or recur automatically for repeat jobs) — a 2-minute supervisor task, since parts, routings and cycle times are already in the master.

- **Part master upload:** part number, description, customer name, part family, routing, cycle time(s), margin (₹ or L/M/H band). One Excel sheet (Template T2). If the plant has 132 part numbers, the system finds 132 — nobody types them.
- **Customer data:** customer NAME per part, nothing more. No addresses, no contacts. The name exists solely so traceability can answer 'which customer received parts from this lot'.
- **People:** two short lists, no org chart. (1) Alert recipients — the 5–8 people who receive escalations and reports: name, role tag, WhatsApp number. (2) Operator names — a plain list for job tagging; names only, no logins for operators at launch.
- **Formulas:** nothing new to collect. Every formula is defined once in Section 8. Template T4 maps each input cell to the formula that consumes it, so every number on every screen traces back to one template cell and one formula.

Onboarding start-line test: if a plant can hand over T1 and T2 filled (or their existing Excel equivalents for us to reshape), the system can go live the same day in Basic costing mode. Everything else can be refined while running.

14.2 How the stages are monitored: Machining → Assembly → Quality → Dispatch

Each stage is an operation on the part's routing (Section 3). The same lot ID travels through all of them, and each stage records one thing: quantity through, and any stops. Dispatch is deliberately included as the final operation — confirming quantity despatched against quantity produced closes the loop and catches losses between QC and the gate.

Stage	What is recorded there	By whom	Device
Op10 Machining	Hourly counts per machine; downtime events with reasons	Zone supervisor	Shared zone tablet (14.5)
Op20 Assembly	Line output count (hourly); line stops with 'which station' as detail	Line leader	One end-of-line tablet
Op30 Quality	Good / rejected with defect codes, per lot	Inspector	Bench tablet or phone
Op40 Dispatch	Quantity despatched per lot, vehicle/challan reference (one field)	Dispatch clerk	Dispatch-desk device

The system therefore always knows, per lot: 500 through machining, 430 through assembly (70 in WIP), 380 passed quality (20 rejected, costed), 380 despatched. Where quantities disagree beyond tolerance, it flags — that gap is where parts silently disappear in most plants today.

14.3 The mid-job part change — the Change Job event

A station running Job A (Part AK-447) is forced to switch to Part BX-102 mid-shift by an urgent order. How is this recorded without corrupting either job's data? The switch is itself an event, and the active job defines every tag:

1. Supervisor taps 'Change Job' on that station. Job A pauses — its quantities, downtime and quality records to this second remain intact and locked to Job A.
2. The changeover clock starts automatically. The gap between 'Job A paused' and the first entry on Job B is recorded as Setup/Changeover downtime, costed like any other downtime. Changeover time — a loss most plants never measure — is captured for free.
3. Supervisor selects Job B from today's list (or creates it in 30 seconds). From this instant, every count, stop and rejection on this station tags to Job B / Part BX-102 automatically.
4. Job A resumes later — same or different machine. The system tracks split quantities across stations and sessions (300 pcs Monday on CNC-3, 200 pcs Tuesday on CNC-5) under one job order, one lot lineage.

Why this can never mis-record a part: entries do not carry a part number typed by a human — they inherit it from whichever job is active on that station at that timestamp. The only way to change the part is the Change Job event, which is itself timestamped and logged. The timeline is unambiguous by construction.

14.4 How costs are collected at scale — 100 machines without 100 headaches

With 100 machines, neither extreme works: 100 individual rates is an onboarding death march, and one plant-wide rate distorts the loss ranking. The answer is cost families:

- Group machines into 5–10 families by type and size (CNC turning, VMC, presses ≤100T, presses >100T, welding, assembly stations). One rate per family — 100 machines typically collapse to 6–8 numbers.
- Individual override only for exceptional machines (the imported 5-axis, the bottleneck line) — usually 2–5 per plant.
- A single plant-wide rate remains the Basic-mode floor for very small plants (≤10 similar machines).

The three-rung ladder when data doesn't exist (small plants):

Rung	Method	Accuracy	When
1. Defaults	Pre-loaded rate library per machine class, from shopfloor benchmarks (e.g. CNC turning ₹350–450/hr machine-only). Entered as flagged estimates.	±30%	Day 1 — nobody blocked
2. Derived	Reconstructed from what exists: wage register ÷ heads ÷ hours (labour); electricity bill × machine share (power); old registers (cycle times).	±15–20%	Week 1
3. Measured	Time studies for cycle times; accountant session for Standard-mode rates.	±5–10%	Weeks 2–6, live

The defaults library deserves a note: a curated, field-validated rate library per machine class for Indian MSME plants does not exist anywhere in the market. Built deliberately from onboarding after onboarding, it becomes both an onboarding accelerator and a proprietary asset competitors cannot download.

14.5 Tablet topology — where the devices physically go

The rule: devices follow people, not machines. In manual mode there is no tablet per machine — that is a hardware-phase concept arriving with sensors. The person who logs events is the supervisor of a zone, so the device belongs to the zone.

Area	Devices	Rule of thumb
Machining sections	1 shared tablet per supervisor zone (≈8–12 machines). Supervisors' own Android phones work identically — the app is a PWA, so BYOD is fully supported.	1 device per zone/supervisor
Assembly line (paced)	The LINE is the unit. ONE tablet at end-of-line: hourly output + line stops, with 'which station stopped us' as the reason detail.	1 device per line
Assembly stations w/ buffers	Only if stations genuinely run independently, treat each buffered station like a machine and apply the zone rule.	Zone rule applies
Quality	Inspector's bench tablet or phone	1 per inspection point
Dispatch	One device at the dispatch desk	1

A typical 20-machine plant with one assembly line therefore runs on 2–3 shared tablets plus personal phones — roughly ₹20–30K of Android hardware, or near zero with BYOD. Per-station granularity and per-machine automatic capture is precisely what the sensor phase adds later — on the same platform, no change above the data layer.

14.6 What the AI Coach needs — inputs and readiness tiers

The Coach requires no data beyond what Factory Core already captures. If a proposed Coach feature needs a new input, the feature is redesigned or rejected. Its fuel is the accumulated daily data; its configuration is only the alert recipients and thresholds already set in onboarding.

Input the Coach consumes	Source	Used for
Downtime events (machine, reason, duration, ₹, timestamps)	Core, daily	Recurrence patterns, risk windows, priority ranking
Production counts vs targets	Core, hourly	Performance drift, target-miss patterns
Quality records (defects, machines, lots)	Core, per batch	Defect trends, machine-quality correlation
Actions and their outcomes (closed, verified, reopened)	Actions engine	What was already tried; nagging unresolved items
Cost parameters and bottleneck flags	Onboarding (T1, T4)	Everything ranked in ₹, honestly
Shift calendar and people/roles	Onboarding (T5, T7)	Who gets told, when, by name

Capability tiers — the Coach never speaks beyond its evidence. These tiers are enforced in the product, not promised in the pitch — the Coach states its own confidence and data basis in every briefing:

Tier	Data condition	What the Coach does
0	Days 1–10	Silent. Escalations run; no coaching. Saying nothing beats guessing.
1	~10 shifts of complete data	Rule-based briefings: biggest loss yesterday; same reason 3× in 5 shifts with no Action; Actions overdue.
2	4–6 weeks	Pattern briefings: breakage intervals, time-of-shift risk windows, cost-ranked prevention actions with named owners.

Tier	Data condition	What the Coach does
3	12+ weeks	Deeper patterns: day-of-week effects, job-specific problems, correlation suggestions (PM gaps → breakdowns) — always as suggestions with evidence shown.

Section 15 — The Onboarding Template Pack (T1–T7)

This is the ingestion contract for the development team: seven Excel templates, every column defined. If it is not in these templates, the system does not need it at onboarding. Build the parsers strict-first (exact columns, clear error messages); tolerant parsing of messy files is a later enhancement. Together the pack is under thirty columns — fillable by a plant in half a day, mostly by handing over files they already have.

T1 — Machines & Stations (one row per machine/station)

Field	Example
Station ID	CNC-3
Station name	CNC Turning 3
Type / family	CNC turning
Section (Machining / Assembly / Quality / Dispatch)	Machining
Bottleneck? (Y/N — set on the floor walk)	Y
Cost family (links to T4)	CNC-family

T2 — Part Master & Routing (one row per part)

Field	Example
Part number	AK-447
Description	Bracket Assembly LH
Customer name	(OEM tier-1 name)
Part family	Brackets
Routing: ordered operations → station family	Op10 CNC-family → Op20 Assembly-L1 → Op30 QC → Op40 Dispatch
Contribution: ₹/part OR band (L/M/H)	₹38 (or 'M')

T3 — Cycle Times (one row per part-operation)

Field	Example
Part number	AK-447
Operation	Op10
Station family	CNC-family
Cycle time (min/part)	2.4
Source: route card / derived / measured	route card

T4 — Cost Inputs

Field	Example
A. Per cost family: monthly depreciation, power share, tooling, maintenance; available hours → machine rate	CNC-family: → ₹420/hr
B. Per section: monthly wage bill, headcount, hours → labour rate	Machining: → ₹180/hr

Field	Example
C. Plant: monthly overhead total, total available machine hours → overhead rate	→ ₹210/hr
D. Margins: per part family, ₹ value or band values agreed with owner	Brackets: L=₹15 M=₹35 H=₹60

T5 — People (two short lists)

Field	Example
A. Alert recipients: name, role tag, WhatsApp number, escalation level	Suresh K., Maintenance, +91…, L2
B. Operators: names only (for job-tagging dropdowns)	Kumar M.; Ravi S.; …

T6 — Reason & Defect Codes

Field	Example
Downtime codes: 18 defaults pre-filled with R/S tags — plant ticks, edits, adds (max ~20)	Tool Breakage (R), Material Wait (R), Power Cut (S)…
Defect codes: 10–15 per part family	Burr, Undersize, Rust spot, Tool mark…

T7 — Shifts & Targets

Field	Example
Shifts: name, start, end, breaks, weekly off	Shift A, 07:00–15:30, 30+15 min, Sun
Daily production target per line/machine group	Line 2: 380 parts/shift

> **Developer note:** T1+T2 are the go-live minimum (Basic costing via defaults). T3 and T4 upgrade accuracy; T5 activates escalation and coaching; T6 has working defaults out of the box; T7 takes five minutes. Every screen, formula and Coach behaviour in this specification traces back to a cell in this pack — there are no hidden inputs.

Section 16 — What the Development Team Needs to Start

This section is the checklist to begin building. If every item here is available, the team can start Stage 1 without waiting on anyone. It is grouped into: data to obtain, formulas already fixed, templates to implement, external setup, and the open decisions that must be resolved as the build proceeds.

16.1 Data to obtain (from one or two real pilot plants)

The build should be designed against real plant data, not invented examples. From an early plant, collect:

- One filled T1 (machine/station list) and T2 (part master + routing) — the go-live minimum.
- Real downtime reason codes as the plant actually uses them, each tagged Recoverable or Structural (T6).
- Real defect codes per part family (T6).
- Cost inputs for at least Basic mode: one blended machine rate per cost family, one labour rate per section, one plant overhead rate, and contribution bands per part family (T4).
- Cycle times for 3–4 key parts, with the source flagged (route card / derived / measured) (T3).
- Shift patterns and daily production targets per line (T7).
- Alert recipients (name, role tag, WhatsApp number) and an operator name list (T5).
- One week of real manual downtime + production + quality logs (from the paper-first exercise) — the dataset to build and test the cost engine against.

16.2 Formulas already fixed (implement exactly once)

These are locked in Section 8 and must be implemented in a single shared calculation module every screen, report and Copilot answer draws from:

- Downtime event cost = (D÷60) × (MCR + LCR + OHR) + ⌊D÷CT⌋ × CM × BF
- Blended machine rate (Basic) and MCR / LCR / OHR (Standard) — as defined in 8.1.
- Scrap cost, rework cost (never both for one part), and Recoverable loss aggregation.
- Bottleneck factor rules (8.2): BF = 1.0 default / bottleneck / whole-line-stop; 0–0.5 for buffered non-bottlenecks; machine+labour+overhead always full.
- OEE = A × P × Q, plus Availability, Performance, Quality, Productivity, Downtime %, Schedule adherence (8.2 tables).
- PPM, First Pass Yield, MTBF, MTTR, Throughput time (multi-stage).
- Recovered Profit (per Action and per plant), Running ROI, Annualised projection — with the baseline-freeze and 2-week sustain rules.

16.3 Templates and screens to implement first (Stage 1)

- The seven ingestion templates T1–T7 (Section 15), strict-parse first.
- Screens 1–6 (Section 7): Shift Start, Machine Grid, Downtime Log, Production Entry, Quality Entry, Shift Closure lock.
- The Digital Readiness Questionnaire (scored one-page output).
- Master-file upload + confirm screen (‘Found 132 parts, 19 machines…’).

- The cost engine wired to the formula module, including the bottleneck factor.
- The job-order spine with routing support in the data model from day one.
- Owner end-of-shift report over WhatsApp; offline-first capture with sync.
- **The data-source abstraction** so a future sensor feed enters through the same contract as a manual entry.

16.4 External setup to start now

- **WhatsApp Business API (WATI):** apply immediately — 3–5 week approval in India; escalations and the Copilot depend on it. SMS fallback in the interim.
- Decide the offline sync + conflict-resolution rules before the first pilot (shared tablets, poor Wi-Fi).
- A default reason-code and defect-code library (T6 defaults) and a default machine-class rate library (14.4, Rung 1) — both seedable before the first plant.

16.5 Open decisions to resolve during the build

These are known choices the team will hit; flag them early rather than discovering them mid-build:

Decision	Why it matters	Default if unresolved
Which IoT device per machine class	Determines the eventual sensor data contract; only needed for the hardware phase	Design the data-source abstraction now; defer device choice
Action trigger thresholds per plant	Too low → action fatigue; too high → real losses ignored	>₹2,500 or >30 min, tuned per plant in onboarding
Severity rules that route to PDCA/RCA	Defines which morning-meeting issues escalate (Section 5.3)	Above Action threshold OR safety/quality-escape flag OR 3+ recurrence
Basic vs Standard costing at go-live	Affects onboarding time and accuracy	Basic mode by default; upgrade in place, never migrate
Tablet count per plant (zone vs line)	Affects hardware cost and entry burden	Devices follow people: 1 per zone/line (Section 14.5)
Cross-verification tolerance band	Sets when a count mismatch flags the owner	Tune against the first plant's real reconciliation data

> **The single most important build discipline:** the NOT-building list in Section 9 is as binding as the feature list. Everything the team needs to start Stage 1 is in Sections 4, 6, 7, 8 and 15 — and consolidated here in Section 16.

Closing Summary

FactoryManager AI is one product with six layers, built bottom-up. At launch: a dead-simple capture system a supervisor feeds in seconds per event; a cost engine that turns events into rupee figures an owner acts on daily (with the Lean judgment — bottleneck factors, recoverable-vs-structural — that keeps those figures honest); an Actions engine that routes only material issues through PDCA/RCA by severity to a verified, measured fix and records the four Factory Memory questions on closure; and a Copilot that answers factual questions from day one and grows pattern intelligence with every week of history. Memory accrues from the first event; the three plugins (Compliance/Audit AI, Kaizen AI, Skill Matrix AI) build in parallel on the same Core data; benchmarking waits for 40–50 plants.

It launches software-only, because the value is in the intelligence, not the sensors — while the data layer is designed from day one so hardware plugs into the same slot a supervisor's thumb occupies today. Which IoT device serves which machine is decided later; the build team focuses on the software now. Everything needed to start is in Sections 4, 6, 7, 8, 15 and 16 — and the NOT-building list in Section 9 is as binding as the feature list.